

Bài 3: LỚP VÀ CÁC THÀNH PHẦN CỦA LỚP ĐỐI TƯỢNG

Mục tiêu

Về kiến thức:

- Cấu trúc lớp và khai báo các thành phần của lớp
- Định nghĩa hàm thành phần và cơ chế nạp chồng, viết đè trong Java
- Các thuộc tính kiểm soát truy cập và kế thừa các thành phần của lớp tượng.

Về kỹ năng:

Giúp sinh viên sẽ nắm được cấu trúc, các thành phần của lớp đối tượng trong ngôn ngữ java, hiểu được phương pháp lập trình hướng đối tượng và thiết kế chương trình bằng việc sử dụng các lớp và đối tượng.

3.1. Cấu trúc lớp và khai báo các thành phần của lớp

3.1.1. Định nghĩa lớp

Lớp là mô hình khái quát đặc tính của một họ thực thể vật lý hoặc trừu tượng trong chương trình.

Định nghĩa lớp là đặc tả một kiểu dữ liệu mới và mô tả cách cài đặt kiểu dữ liệu đó.

Mỗi đối tượng là một thể hiện của lớp.

* Cú pháp định nghĩa lớp

```
[<Phạm vi hoặc thuộc tính kiểm soát truy nhập>] class <Tên lớp>  
[extends <Tên lớp cha>] [implements <Tên giao diện>]  
{ <Các thành phần của lớp> }
```

* Giải thích:

Phạm vi: Xác định phạm vi hoặc kiểm soát truy nhập:

- *abstract:* Không thể tạo ra một đối tượng cụ thể của lớp, lớp loại này là khung cơ sở nhất để dẫn xuất ra các lớp con cháu.
- *public:* (công khai) Cho phép sử dụng mọi nơi trong hệ thống.
- *protected:* cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private:* chỉ được sử dụng bên trong lớp, không cho phép truy cập và kế thừa đối với các lớp khác ở trong cùng gói và khác gói.
- *final:* (hằng) nếu nó đứng trước khai báo lớp thì nó không được sử dụng để kế thừa cho các lớp con, nếu nó đứng trước khai báo biến thì nó là “hằng” (chỉ được gán giá trị một lần duy nhất), nếu nó

đứng trước các phương thức thì nó sẽ không được phép viết đè ở các lớp con.

- Nếu không khai báo (public, protected, private) thì gọi là các thành phần “mặc định”, khi đó phạm vi là truy cập và kế thừa là trong gói chứa lớp đó.

tên_lớp_cha: Tên lớp cha được kế thừa

DS_tên_các_giao_diện: Tên các giao diện (được viết phân cách nhau bởi dấu “,”) được phép sử dụng các phương thức và các hằng.

class, extends, implements là các từ khoá.

Những phần trong cặp [và] là tùy chọn(có thể có hoặc có thể không).

<Các thành phần của lớp> bao gồm:

- Các biến dữ liệu thành phần(attributes) : Xác định các thuộc tính (dữ liệu) của lớp, thông thường dùng để mô tả các đặc tính của đối tượng thuộc lớp đó
- Các toán tử tạo lập(constructor): Xác định cách khởi tạo một đối tượng thuộc lớp
- Các phương thức(method): Xác định hành vi của các đối tượng thuộc lớp

3.1.2. Các thuộc tính dữ liệu của lớp

Thuộc tính dữ liệu của lớp là các biến thành phần có thể thuộc kiểu nguyên thủy, kiểu tham chiếu.

* Khai báo:

<Phạm_vi> <Kiểu> <tên_biến>;

<Phạm_vi> <Kiểu> <tên_biến> = <Giá_trị_khởi_tạo>;

Phạm vi: xác định phạm vi truy cập của các đối tượng đến thuộc tính. có thể chọn một trong các phạm vi:

- *static*: Khi một thuộc tính được khai báo static thì khi đó thuộc tính đó là “sở hữu” của tất cả các đối tượng của lớp đó chứ không phải nó là của riêng của từng đối tượng thuộc lớp đó.

- *public*: Có thể truy cập mọi nơi trong hệ thống (các đối tượng khác lớp

có thể truy cập được)

- *protected*: cho phép truy cập trong gói và trong các lớp thuộc gói khác nhưng được kế thừa.
- *private*: của riêng đối tượng chỉ bên trong đối tượng.
- *final*: Hằng (chỉ được khởi tạo giá trị một lần duy nhất)
- *volatile*: là biến được lưu trữ trên bộ nhớ chính, được dùng với luồng kết hợp với từ khóa *synchronized*.

3.2. Định nghĩa hàm thành phần và cơ chế nạp chồng, viết đè trong

Java

3.2.1. Các hàm thành phần của lớp

Hành vi của các đối tượng thuộc một lớp được xác định bởi các hàm thành phần của lớp đó, hành vi thể hiện “vai trò” của đối tượng đó trong hệ thống.

Là các hàm, thủ tục thực hiện các phép xử lý trên các biến thành phần.

* Khai báo

*<Phạm_vi> <Kiểu_giá_tri> <tên_hàm>(< danh sách các tham số hình thức>)
<các_mệnh_đề_throws> { // Thân hàm }*

Phạm_vi:

- *public*: Cho phép truy cập mọi nơi trong hệ thống
 - *private*: Chỉ được truy xuất (gọi) bên trong lớp chứa nó
 - *protected*: lớp đó và lớp con của nó được truy xuất
 - *final*: Không cho phép nạp chồng và ghi đè
 - *static*: Nó có ý nghĩa tương tự như với thuộc tính *static*, thông thường chỉ những phương thức chung và không dùng cho một đối tượng cụ thể thì khi đó phương thức đó nó mới là phương thức tĩnh (*static*).
 - *abstract*: Phương thức trừu tượng (là phương thức “mẫu”, nó chỉ nói lên là nó làm được gì mà không mô tả nó làm như thế nào), do đó nó không cài đặt nội dung ở lớp khai báo nó
 - *native*: dùng cho phương thức khi cài đặt phụ thuộc môi trường trong một ngôn ngữ khác, như C hay hợp ngữ.
 - *synchronized*: dùng trong lập trình đa luồng, nhằm ngăn các tác động của các phương thức khác của các luồng khác khi phương thức đang được thực hiện.
 - Giá trị mặc định là *public*.
- <Kiểu trả lại>* có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại

(kiểu void).

<Danh sách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy.

*** Sử dụng phương thức:**

- Nếu phương thức không phải là static thì cần phải tạo ra một đối tượng cụ thể và gọi phương thức thông qua tên đối tượng như cấu trúc sau:

<tên đối tượng>.<tên_hàm>(< danh sách các tham số hiện thời>)

Khai báo đối tượng: <tên lớp> <tên đối tượng> ;

Khởi tạo giá trị thuộc tính cho đối tượng khi sử dụng Toán tử tạo lập lập(constructor) tự định nghĩa :

<tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >) ;

Khai báo và khởi tạo đối tượng giá trị:

<tên lớp> <tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >) ;

- Nếu phương thức là static thì gọi thông qua tên lớp hoặc thông qua tên đối tượng (nên gọi các phương thức tĩnh thông qua tên lớp)

<tên_lớp>.<tên_hàm_tĩnh> (<danh sách tham số hiện thời >)

- Các tham số trong các lời gọi hàm cung cấp cách thức trao đổi thông tin giữa đối tượng gửi và đối tượng nhận thông điệp.

- Số các tham biến của danh sách hiện thời phải bằng số các tham biến của danh sách tham biến hình thức.
- Kiểu của các tham biến hiện thời phải tương thích với kiểu của tham biến hình thức tương ứng.

Kiểu của tham biến hiện thời	Giá trị được truyền
Các kiểu nguyên thủy	Giá trị kiểu nguyên thủy
Kiểu lớp (class)	Giá trị tham chiếu
Kiểu mảng (array)	Giá trị tham chiếu

Ví dụ1: Tập Demo.java

```
public class Demo {  
    public static void main(String args[]) {  
        int i = 5;  
        if (i == 0) {
```

```

        return;
    }
    output(checkValue(args.length));
}
static void output(int value) {
    System.out.println(value);
}
static int checkValue(int i) {
    if (i > 3) {
        return 1;
    } else {
        return 2;
    }
}
}
}

```

Ví dụ2: Tập Demo2.java

```

public class Demo2 {
    public static void main(String args[]) {
        int i = 5;
        Demo2 d = new Demo2();
        if (i == 0) {
            return;
        }
        output(d.checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    public int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {

```

```

        return 2;
    }
}

```

*** Phương thức và thuộc tính static:** Phương thức và thuộc tính độc lập với đối tượng, có thể sử dụng mà không cần thông qua đối tượng, Phương thức tĩnh: không sử dụng được thuộc tính và phương thức “thông thường” (non-static) mà nó chỉ được gọi đến các phương thức và thuộc tính tĩnh khác

3.2.2 Truyền tham số và gọi phương thức(hàm)

Truyền các giá trị kiểu nguyên thủy

Trong Java, khi gọi một phương thức và truyền các giá trị nguyên thủy cho phương thức, được gọi là truyền tham trị. Việc thay đổi giá trị chỉ có hiệu lực trong phương thức được gọi, không có hiệu lực bên ngoài phương thức.

Các tham số có kiểu dữ liệu nguyên thủy là byte, short, int, long, float, double, boolean, char.

Ví dụ: *Truyền các giá trị nguyên thủy*

```

class KhachHang1 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra một đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}

// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, double gia) {
        gia = gia / 2;
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền bị thay đổi theo
    }
}

```

```
}
```

Truyền các giá trị tham chiếu đối tượng.

Trong Java, khi gọi một phương thức và truyền một tham chiếu cho phương thức, được gọi là truyền tham chiếu. Việc thay đổi giá trị của biến tham chiếu bên trong phương thức làm thay đổi giá trị của nó.

Trong Java, tất các phương thức có tham số là biến có kiểu là các lớp (class) đều là kiểu tham chiếu

```
//KhachHang2.java
```

```
class KhachHang2 { // Lớp khách hàng
```

```
    public static void main(String[] arg) {
```

```
        Banh banhMoi = new Banh(); // Tạo ra một đối tượng (1)
```

```
        System.out.println("Nhoi thit vao banh truoc khi nuong:" +  
banhMoi.thit);
```

```
        nuong(banhMoi); // (2)
```

```
        System.out.println("Thit cua banh sau khi nuong:" + banhMoi.thit);
```

```
    }
```

```
    public static void nuong(Banh banhNuong) { // (3)
```

```
        banhNuong.thit = "Thit vit
```

```
        "; // đổi nhân thành thịt vịt
```

```
        banhNuong = null; // (4)
```

```
    }
```

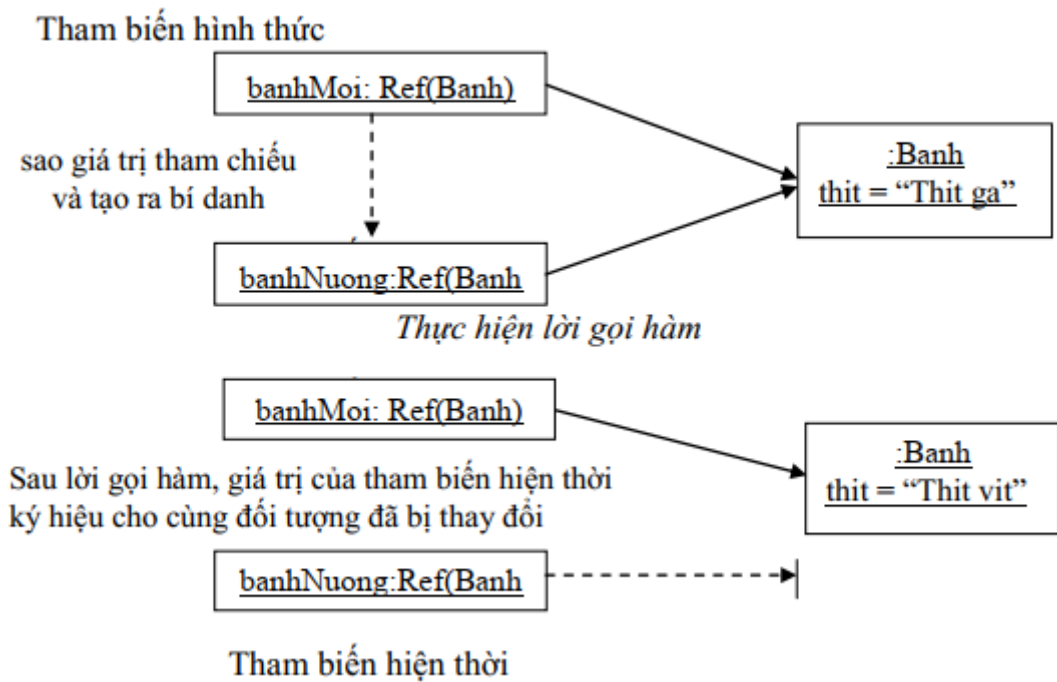
```
}
```

```
class Banh { // Lớp Banh (5)
```

```
    String thit = "Thit ga
```

```
    "; // Qui định của hãng làm nhân bánh bằng thịt gà
```

```
}
```



Truyền tham chiếu: đối với đối tượng thì nội dung của tham chiếu (LValue) được copy lên stack.

Truyền đối số trong dòng lệnh

```

class Pass {
    public static void main(String parameters[]) {
        System.out.println("This is what the main method received");
        System.out.println(parameters[0]);
        System.out.println(parameters[1]);
        System.out.println(parameters[2]);
    }
}
  
```

Biên dịch chương trình: **javac PassArgument.java**

Thực thi chương trình với dòng lệnh: **java PassArgument A 123 B1**

Sẽ thu được trên màn hình kết quả:

```

This is what the main method received
A
123
B1
  
```

Các tham biến final

Tham biến loại này được gọi là biến cuối “trắng”, nghĩa là nó không được khởi tạo giá trị (là trắng) cho đến khi nó được gán một trị nào đó và khi đã được gán trị thì giá trị đó là cuối cùng, không thay đổi được.

Ví dụ *Sử dụng tham biến final*

```
//KhachHang3.java
class KhachHang3 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra 1 đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}

// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, final double gia) { // (1)
        gia = gia / 2.0; // (2)
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền vẫn bị thay đổi theo
    }
}
```

3.2.3. Cơ chế nạp chồng, viết đè trong Java

Nạp chồng

Nạp chồng là cho phép sử dụng cùng tên hàm nhưng định nghĩa nhiều nội dung thực hiện khác nhau.

Các hàm nạp chồng phải khác nhau về danh sách tham biến (khác nhau về kiểu tham biến hoặc số lượng tham biến hay là có định danh khác nhau)

Ví dụ: Xây dựng lớp các hình học, trong đó xây dựng các hàm nạp chồng tính diện tích

```
public class HìnhHoc {
    static int dienTich(int a) {
```

```

        System.out.println("Dien tich hinh vuong: ");
        return a * a;
    }
    static int dienTich(int a, int b) {
        System.out.println("Dien tich chu nhac: ");
        return a * b;
    }
    static double dienTich(int a, double b) {
        System.out.println("Dien tich tron: ");
        return a * a * b;
    }
    public static void main(String[] arg) {
        int a = 3;
        int b = 4;
        double c = 3.14;
        System.out.println(" " + dienTich(a));
        System.out.println(" " + dienTich(a, c));
        System.out.println(" " + dienTich(a, b));
    }
}

```

JDK API đã xây dựng rất nhiều hàm được nạp chồng.

Lưu ý là danh sách tham biến của các hàm nạp chồng phải khác nhau về số lượng hoặc về thứ tự các kiểu của các tham biến.

Ví dụ:

Hàm	Định danh
public void methodA(int a, double b){/* ...*/}	methodA(int, double)
public int methodA(int a){return a}	methodA(int)
public int methodA(){return 1} // (3)	methodA()

public long methodA(double a,int b){return a*b}	methodA(double, int)
public long methodA(int a, double b){return a} //NoOK	methodA(int, double) không đúng do định danh giống (1")

Viết chồng (viết đè)

Một lớp con có thể viết đè, thay đổi nội dung thực hiện của những hàm thừa kế từ một lớp cha gọi là viết đè các hàm thành phần.

Khi những hàm này được gọi để thực thi đối với những đối tượng của lớp con thì nội dung thực hiện được định nghĩa ở lớp con sẽ được thực thi.

Ví dụ:

- Xây dựng lớp Meo có hàm thành phần speak()
- Xây dựng lớp MeoCon kế thừa lớp Meo và viết đè hàm speak()
- Gọi hàm speak() của lớp MeoCon
- Gọi hàm speak() của lớp Meo đối với đối tượng thuộc lớp MeoCon

```
class Meo{
    protected static void speak(){
        System.out.print("Meo Meo!"); }
}
class MeoCon extends Meo{
    protected static void speak(){
        System.out.print("Miu Miu!"); }
    public static void main(String []arg){
        Meo meo=new Meo();
        MeoCon meocon=new MeoCon();
        MeoCon.speak();
        System.out.println();
        meo.speak();
        System.out.println(); }
}
```

* Lưu ý:

- Định nghĩa mới của hàm viết đè phải có cùng định danh (tên gọi và danh sách tham biến) và cùng kiểu trả lại giá trị.
- Định nghĩa mới của hàm viết đè trong lớp con chỉ có thể xác định tất cả hoặc tập con các lớp ngoại lệ được kể ra trong mệnh đề cho qua ngoại lệ
- Các hàm final, static không được phép viết đè.
Phân biệt rõ hai cơ chế viết đè và nạp chồng là khác nhau trong Java.
- Viết đè yêu cầu cùng định danh hàm (cùng tên gọi, cùng danh sách tham số) và cùng kiểu trả lại đã được định nghĩa tại lớp cha.
- Nạp chồng yêu cầu khác nhau về định danh, nhưng giống nhau về tên gọi của hàm, vì thế chúng sẽ khác nhau về số lượng, kiểu, hay thứ tự của các tham biến.
- Hàm có thể nạp chồng ở trong cùng lớp hoặc ở các lớp con cháu.
- Từ những lớp con khi muốn gọi tới các hàm ở lớp cha mà bị viết đè thì phải gọi qua toán tử đại diện cho lớp cha, đó là super(). Đối với hàm nạp chồng thì lại không cần như thế. Lời gọi hàm nạp chồng được xác định thông qua danh sách các đối số hiện thời so sánh với đối số hình thức để xác định nội dung tương ứng.

3.2.4. Nạp chồng các toán tử tạo lập

Giống như các hàm thành phần, các toán tử tạo lập có thể nạp chồng với nhiều nội dung thực hiện khác nhau.

Khi tạo lập đối tượng sử dụng toán tử tạo lập nào thì toán tử đó được gọi.

Ví dụ:

```
class BongDen{
    // Biến thành phần (1)
    private int soWatts;
    private boolean batTat;
    private String viTri;
    // Định nghĩa toán tử tạo lập số 1 (2)
    BongDen(){
        this(40, true);
        System.out.println("Toán tử số 1"); }
    // Định nghĩa toán tử tạo lập số 2 (3)
```

```

    BongDen(int w, boolean s){
        this(w, s, "XX");
        System.out.println("Toán tử số 2"); }
//Định nghĩa toán tử tạo lập số 3 (4)
    BongDen(int soWatts, boolean batTat, String viTri){
        this.soWatts = soWatts;
        this.batTat = batTat;
        this.viTri = new String(viTri);
        System.out.println("Toán tử số 3"); }
}

class DenTuyp extends BongDen {
    private int doDai;
    private int mau;
    DenTuyp(int leng, int colo){
        this(leng,colo,100,true, "Chua biet");
    }
    DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){
        super(soWatt, bt, noi);
        this.doDai = leng;
        this.mau = colo;
    }
}

public class NhaKho{
    public static void main(String args[]){
        System.out.println("Tao ra bong đèn tuyp");
        DenTuyp d = new DenTuyp(20, 5);
    }
}

```

3.2.5. Cơ chế che bóng của các biến

Về nguyên tắc, một lớp con không được phép viết đè các biến thành phần của lớp cha, nhưng có thể che khuất chúng tương tự như biến cục bộ trong lớp con.

Ví dụ *Viết đè và nạp chồng các hàm thành phần*

// *KhachHang.java*

```

import java.io.*;
class Den {
    protected String loaiHoaDon = "Hoa don nho: "; // (1)
    protected double docHoaDon(int giaDien) throws Exception { // (2)
        double soGio = 10.0, hoaDonNho = giaDien * soGio;
        System.out.println(loaiHoaDon + hoaDonNho);
        return hoaDonNho;
    }
}

class DenTuyp extends Den {
    public String loaiHoaDon = "Hoa don lon: "; // Bị che bóng (3)
    public double docHoaDon(int giaDien) throws Exception { // Viết đè hàm
(4)
        double soGio = 100.0;
        hoaDonLon = giaDien * soGio;
        System.out.println(loaiHoaDon + hoaDonLon);
        return hoaDonLon;
    }

    public double docHoaDon() {
        System.out.println("Khong co hoa don!");
        return 0.0;
    }
}

public class KhachHang {
    public static void main(String args[]) throws Exception { // (6)
        DenTuyp den1 = new DenTuyp(); // (7)
        Den den2 = den1; // (8)
        Den den3 = new Den(); // (9)
        // Gọi các hàm đã viết đè
        den1.docHoaDon(1000); // (10)
        den2.docHoaDon(1000); // (11)
        den3.docHoaDon(1000); // (12)
    }
}

```

```
// Truy nhập tới các biến thành phần đã bị viết đè (bị che bóng)
System.out.println(den1.loaiHoaDon); // (13)
System.out.println(den2.loaiHoaDon); // (14)
System.out.println(den3.loaiHoaDon); // (15)

// Gọi các hàm nạp chồng
den1.docHoaDon();
}
}
```

Kết quả thực hiện của chương trình KháchHang:

Hóa đơn lớn: 100000.00

Hóa đơn lớn: 100000.00

Hóa đơn nhỏ: 10000.00

Hóa đơn lớn:

Hóa đơn nhỏ:

Hóa đơn nhỏ:

Không có Hóa đơn!

3.3. Các thuộc tính kiểm soát truy nhập các thành phần của lớp

Khi xây dựng một lớp ta có thể hạn chế sự truy cập đến các thành viên của lớp, từ một đối tượng khác. Ta tóm tắt qua bảng sau:

Từ khoá	Truy cập trong chính lớp đó	Truy cập trong lớp con cùng gói	Truy cập trong lớp con khác gói	Truy cập trong lớp khác cùng gói	Truy cập trong lớp khác khác gói
private	X	-	-	-	-
protected	X	X	X	X	-
public	X	X	X	X	X
default	X	X	-	X	-

Trong bảng trên thì X thể hiện cho sự truy cập hợp lệ còn – thể hiện không thể truy cập vào thành phần này.

***Ví dụ:** Các thành phần private

Các thành viên private chỉ có thể sử dụng bên trong lớp, ta không thể truy cập các thành viên private từ bên ngoài lớp này.

```

class Alpha {
    private int iamprivate;
    private void privateMethod() {
        System.out.println("privateMethod");
    }
}

class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iamprivate = 10; // không hợp lệ
        a.privateMethod(); // không hợp lệ
    }
}

```

* Ví dụ: Các thành phần public

Các thành viên public có thể truy cập từ bất cứ đâu, ta sẽ xem ví dụ sau:

```

Package Greek;

public class Alpha {
    public int iampublic;
    public void publicMethod() {
        System.out.println("publicMethod");
    }
}

package Roman;
import Greek.*;

class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iampublic = 10; // hợp lệ
        a.publicMethod(); // hợp lệ
    }
}

```

* Ví dụ các thành phần có mức truy xuất gói:

Khi ta khai báo các thành viên mà không sử dụng một trong các từ public, private, protected thì java mặc định thành viên đó có mức truy cập gói.

Ví dụ:

```
package Greek;

class Alpha {

    int iampackage;

    void packageMethod() {

        System.out.println("packageMethod");

    }

}

package Greek;

class Beta {

    void accessMethod() {

        Alpha a = new Alpha();

        a.iampackage = 10;// hợp lệ

        a.packageMethod();// hợp lệ

    }

}
```

3.4. Toán tử tạo lập đối tượng

Đối tượng(Object) là một thể hiện của một lớp(Class). Lớp là một mẫu hoặc thiết kế từ đó các đối tượng được tạo ra. Vì vậy, đối tượng là các thể hiện của một lớp.

Toán tử tạo lập đối tượng được sử dụng để tạo ra thể hiện của lớp, nghĩa là đặt các giá trị khởi tạo cho đối tượng khi một đối tượng được tạo ra bằng toán tử new

Có 2 kiểu của constructor:

- Constructor mặc định (không có tham số truyền vào – default constructors).
- Constructor tham số (parameterized constructors), ta có thể tự định nghĩa toán tử tạo lập cho một lớp đối tượng.

3.4.1. Khai báo toán tử tạo lập

* Khai báo:

[phạm vi] <tênlớp>(<danh sách các tham biến>) { //Phần thân}

Phạm_vi có thể là: public, protected, private

Phần thân là các lệnh khởi tạo giá trị các thuộc tính của đối tượng, là thành phần tùy chọn.

Danh sách các tham số : tùy chọn

* Phân loại:

- Toán tử tạo lập mặc định không tường minh: **Tên_Lớp()** {}

Khi định nghĩa một lớp mà không xây dựng toán tử tạo lập nào thì hệ thống sẽ cung cấp toán tử tạo lập mặc định không tường minh.

- Toán tử tạo lập mặc định tường minh:

```
TênLớp(<danh sách tham biến hình thức>) {  
    tên thuộc tính= giá trị;  
    ..... }
```

Giá trị của các biến thành phần được khởi tạo không mặc định bằng các phép gán trong thân của toán tử bằng các giá trị được truyền vào.

Ví dụ: Minh họa một phương thức khởi tạo của lớp Person bằng cách gán giá trị cho các thuộc tính tên và tuổi.

```
class Person {  
    public String name;  
    public int age;  
    // Phương thức tạo lập mặc định không tường minh  
    //public Person(){}  
    // Phương thức tạo lập mặc định tường minh  
    public Person(){  
        name = "";  
        age = 16;  
    }  
    // Phương thức tạo lập không mặc định  
    public Person(String name1, int age1){  
        name = name1;  
        age = age1;  
    }  
    // Phương thức tạo lập không mặc định  
    public Person(String name1, int age1){
```

```

        name = name1;
        age = age1;
    }
    /*public Person(String name, int age){
        this.name = name;
        this.age = age;
    }*/
    public void show(){
        System.out.println( name + " is " + age + " years old!");
    }
}

```

3.4.2. Toán tử this() và super()

Toán tử tạo lập this() Toán tử tạo lập này được sử dụng để tham chiếu đến các thành phần(là các thuộc tính và phương thức) của lớp hiện thời	Toán tử tạo lập super() Được sử dụng tham chiếu đến các thành phần thuộc lớp cha trực tiếp
<pre> class BongDen{ // Biến thành phần (1) private int soWatts; private boolean batTat; private String viTri; // Toán tử tạo lập số 1 (2) BongDen(){ this(40, true); System.out.println("Toán tử số 1"); } // Toán tử tạo lập không mặc định số 2 (3) BongDen(int w, boolean s){ this(w, s, "XX"); System.out.println("Toán tử số 2"); } // Toán tử tạo lập không mặc định số 3 (4) </pre>	<pre> class BongDen{ // Biến thành phần (1) private int soWatts; private boolean batTat; private String viTri; // Toán tử tạo lập số 2 (2) BongDen(){ this(40, true); System.out.println("Toán tử số 1"); } // Toán tử tạo lập không mặc định số 2 (3) BongDen(int w, boolean s){ this(w, s, "XX"); System.out.println("Toán tử số 2"); } </pre>

<pre> BongDen(int soWatts, Boolean batTat, String viTri){ this.soWatts = soWatts; this.batTat = batTat; this.viTri = new String(viTri); System.out.println("Toán tử số 3"); } } </pre>	<pre> } // Toán tử tạo lập không mặc định số 3 (4) BongDen(int soWatts, Boolean batTat, String viTri){ this.soWatts = soWatts; this.batTat = batTat; this.viTri = new String(viTri); System.out.println("Toán tử số 3"); } } </pre>
<pre> public class NhaKho{ public static void main(String args[]){ BongDen d1=new BongDen();// OK BongDen d2 = new BongDen(100, true, "Nha bep"); // OK BongDen d3 = new BongDen(100, true); //OK } } </pre>	<pre> class DenTuyp extends BongDen { private int doDai; private int mau; DenTuyp(int leng, int colo){ // (5) this(leng, colo, 100, true, "Chua biet"); } DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){ // (6) super(soWatt, bt, noi); this.doDai = leng; this.mau = colo; } } public class NhaKho{ public static void main(String args[]){ System.out.println("Tao ra bong den tuyt"); DenTuyp d = new DenTuyp(20,5); } } </pre>

+ Bài tập giải mẫu

Viết chương trình xây dựng lớp Diem gồm:

-Các thuộc tính double x, y tương ứng là hai giá trị hoành độ và tung độ và các phương thức:

Các phương thức tạo lập:

```
public Diem(){  
    public Diem(double x,double y){x=x1;y=y1}
```

Các phương thức :

```
public void inTTDiem(){ // in giá trị x,y ra màn hình }  
public double tinhKhoangCach(Diem d) {tính khoảng cách giữa điểm  
    hiện thời và đối số d}
```

Cài đặt lớp **Main** thực hiện tạo ra hai điểm $D1=new\ Diem(3,7); D2= new\ Diem(9,3)$ và in khoảng cách của hai điểm trên.

```
import java.util.Scanner;  
public class Diem {  
    double x,y;  
    Diem() {  
    }  
    public void nhapTTDiem()  
    {Scanner input= new Scanner(System.in);  
    System.out.print("Nhập toạ độ x:= ");  
    x=input.nextDouble();  
    System.out.print("Nhập toạ độ y:= ");  
    y=input.nextDouble();  
    }  
    public void inTTDiem()  
    {System.out.println("Toạ độ X:"+x+"Toạ độ Y: "+y);  
    }  
    Diem(double x1,double y1)  
    { x=x1;y=y1;  
    }  
}
```

```

        public double tinhKhoangCach(Diem d1)
        { return Math.sqrt((x-d1.x)*(x-d1.x)+(y-d1.y)*(y-d1.y));
        }
    }
}

```

+ Các bài tập thực hành

Bài 1: Viết chương trình:

1. Xây dựng lớp **Nguoi** gồm:

- Các thuộc tính chung : hoTen, diaChi, namSinh
- Các phương thức:

Phương thức: public void nhapTT() { // nhập thông tin cho Nguoi }

Phương thức: public void inTT() { // in thông tin là các giá trị thuộc tính cho Nguoi }, public void nhapTT() { // nhập thông tin từ bàn phím cho các thông tin của Nguoi }

2. Xây dựng lớp **Main** chứa phương thức main(): Cho phép nhập thông tin của 2,3 người và hiển thị thông tin của người vừa nhập ra màn hình.

Bài 2: Thư viện của trường đại học KHTN có nhu cầu cần quản lý việc mượn sách. Sinh viên đăng ký và tham gia mượn sách thông qua các thẻ mượn mà thư viện đã thiết kế.

-Với mỗi thẻ mượn, có các thông tin sau: số phiếu mượn, ngày mượn, hạn trả, số hiệu sách, và các thông tin riêng về mỗi sinh viên đó.

-Các thông tin riêng về mỗi sinh viên đó bao gồm: Họ tên, năm sinh, tuổi, lớp.

Hãy xây dựng các lớp: SinhVien để quản lý các thông tin riêng về mỗi sinh viên, lớp TheMuon để quản lý việc mượn sách của mỗi đọc giả.

Viết chương trình nhập và hiển thị thông tin của một thẻ mượn.

Bài 3: Để quản lý các biên lai thu tiền điện, người ta cần các thông tin như sau: Với mỗi biên lai, có các thông tin sau: thông tin về hộ sử dụng điện, chỉ số cũ, chỉ số mới, số tiền phải trả của mỗi hộ sử dụng điện

Các thông tin riêng của mỗi hộ sử dụng điện gồm: Họ tên chủ hộ, số nhà, mã số công tơ của hộ dân sử dụng điện.

1. Hãy xây dựng các lớp: **KhachHang** để lưu trữ các thông tin riêng của mỗi hộ sử dụng điện, lớp **BienLai** để quản lý việc sử dụng và thanh toán tiền điện của các hộ dân. Xây dựng các phương thức nhập, và hiển thị thông tin của mỗi hộ sử dụng điện.

2. Cài đặt chương trình thực hiện các công việc sau:

- + Nhập vào các thông tin cho 2 hộ sử dụng điện
- + Tính tiền điện phải trả cho mỗi hộ dân, biết rằng tiền phải trả được tính theo công thức sau: $Số tiền phải trả = (Số mới - số cũ) * 850000$.

+ Các bài thực hành mức độ nâng cao.

Bài 1: Hãy xây dựng lớp DaGiac gồm có:

Các thuộc tính:

- + Số cạnh của đa giác
- + Mảng các số thực chứa kích thước các cạnh của đa giác

Các phương thức:

- + Nhập vào giá trị cho các cạnh của đa giác
- + Tính chu vi của đa giác
- + Hiện thị giá trị các cạnh của đa giác.

Xây dựng lớp TamGiac kế thừa từ lớp DaGiac, thực hiện ghi đè các phương thức của lớp TamGiac lên các phương thức của lớp DaGiac.

Xây dựng hàm main() nhập kích thước của một tam giác và hiện thị chu vi, diện tích của tam giác đó.

Bài mẫu

Bài 1 : Tạo 1 lớp trừu tượng (abstract) Person chứa các thông tin về người, tạo tiếp 2 lớp Students và Teachers kế thừa lớp Person, tạo lớp Execute chứa hàm main để chạy chương trình.

```
import java.io.Console;
import java.util.Scanner;
abstract class Person {
    //cai nay goi la cac property hay state-thuoc tinh cua doi tuong
    String hoten;
    int age;
    String diachi;
    int luong;
    //cac constructor
    Person() {
```

```

    }
    public Person(int age) {
        this.age = age;
    }
    //cac method hay behavior-hanh vi cua doi tuong
    public void Nhap() {
        Console input = System.console();
        hoten = input.readLine("Nhap ho ten:");
        diachi = input.readLine("Nhap dia chi:");
    }
    public abstract void In();
    public abstract int Tinhluong();
}

class Students extends Person {
    int MaSV, Malop;
    public Students(int age) {
        super(age);
    }
    public Students() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma SV: ");
        MaSV = input.nextInt();
        System.out.println("Nhap Ma Lop: ");
        Malop = input.nextInt();
    }

    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
    }
}

```

```

        System.out.println(MaSV);
        System.out.println(Malop);
    }
    public int Tinhluong() {
        return 150000;
    }
}

class Teachers extends Person {
    int Makhoa;
    public Teachers(int age) {
        super(age);
    }
    public Teachers() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma khoa: ");
        Makhoa = input.nextInt();
    }
    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
        System.out.println(Makhoa);
    }
    public int Tinhluong() {
        return 500000;
    }
}

public class Execute {
    public static void main(String args[]) {
        Students st = new Students();
    }
}

```

```

        st.Nhap();
        st.In();
        st.luong = st.Tinhluong();
        Teachers tc = new Teachers();
        tc.Nhap();
        tc.In();
        tc.luong = tc.Tinhluong();
    }
}

```

Bài 2 : **Shape** là lớp trừu tượng, trình triển khai của nó được cung cấp bởi lớp **Rectangle** và lớp **Circle**. Hai lớp này kế thừa lớp trừu tượng **Shape**.

```

// lop truu tuong Shape
abstract class Shape {
    abstract void draw();
}

//Trong tinh huong nay, trinh trien khai duoc cung cap boi ai do, vi du: nguoi su
dung cuoi cung nao do
class Rectangle extends Shape {
    void draw() {
        System.out.println("Ve hình chu nhật");
    }
}

class Circle1 extends Shape {
    void draw() {
        System.out.println("Ve hình tròn");
    }
}

//Trong tinh huong nay, phuong thuc duoc goi boi lap trinh vien hoac nguoi dung
class TestAbstraction1 {
    public static void main(String args[]) {
        Shape s = new Circle1(); //Trong tinh huong nay, doi tuong duoc cung cap
thong qua phuong thuc, chang han nhu getShape()
    }
}

```

```

s.draw();
}
}

```

+ Các bài tập thực hành

Bài 1 : Định nghĩa giao diện **IVehicle** gồm có các hằng và phương thức :

```

interface IVehicle {
    // khai báo và khởi tạo hằng
    static final String STATEID = "LA-09";
    //phương thức trừu tượng khởi động xe
    public void start();
    // phương thức trừu tượng tăng tốc xe
    public void accelerate(int speed);
    // phương thức trừu tượng hãm phanh
    public void brake();
    // phương thức trừu tượng dừng xe
    public void stop();
}

```

Xây dựng lớp **TwoWheeler** kế thừa từ **IVehicle** và cài đặt các phương thức và thuộc tính

```

class TwoWheeler implements IVehicle {
    // biến lưu ID của xe
    String ID;
    // biến lưu kiểu xe
    String type;
    // Toán tử tạo lập có tham số để khởi tạo các giá trị nhập vào từ người dùng
    public TwoWheeler(String ID, String type) {
        this.ID = ID;
        this.type = type;
    }
    //ghi đè các phương thức để khởi động xe
    @Override
    public void start() {

```

```

        System.out.println("Starting the " + type);
    }
    // ghi đè phương thức để tăng tốc xe
    @Override
    public void accelerate(int speed) {
        System.out.println("Accelerating at speed:" + speed + " kmph");
    }
    // ghi đè phương thức để hãm phanh
    @Override
    public void brake() {
        System.out.println("Applying brakes");
    }
    // ghi đè phương thức để dừng xe
    @Override
    public void stop() {
        System.out.println("Stopping the " + type);
    }
    // hiển thị thông số của xe
    public void displayDetails() {
        System.out.println("Vehicle No.: " + STATEID + " " + ID);
        System.out.println("Vehicle Type.: " + type);
    }
}

```

Xây dựng lớp **TestVehicle** chứa hàm main khởi tạo đối tượng và test các phương thức:

```

public class TestVehicle {
    public static void main(String[] args) {
        // xác nhận số lượng đối số dòng lệnh
        // tạo thể hiện cho lớp TwoWheeler
        TwoWheeler objBike = new TwoWheeler("20A 22584", "HonDa Civic");
        // gọi các phương thức objBike.displayDetails();
        objBike.start();
    }
}

```

```

        objBike.accelerate(200);
        objBike.brake();
        objBike.stop();
    }
}

```

Bài 2: Viết chương trình xây dựng lớp SoPhuc gồm:

Các thuộc tính riêng gồm: phanThuc, phanAo kiểu double

Các phương thức:

- + Các toán tử tạo lập : SoPhuc(), SoPhuc(float pt, float pa)
- + Phương thức nhập vào một số phức: void nhapSoPhuc()
- + Phương thức hiển thị một số phức: void inSoPhuc()
- + Phương thức cộng hai số phức : SoPhuc congSoPhuc(SoPhuc sp)
- + Phương thức nhân hai số phức: SoPhuc nhanSoPhuc(SoPhuc sp)

Cài đặt chương trình (lớp Main) nhập vào hai số phức A và B, sau đó tính số phức tổng $C1 = A.congSoPhuc(B)$, nhân $C = A.nhanSoPhuc(B)$ rồi hiển thị kết quả ra màn hình như sau:

Tổng: $(a + b.i) + (c + d.i) = (a + c) + (b + d).i$

Tích: $(a + b.i)(c + d.i) = (a.c - b.d) + (b.c + a.d).i$

Chia :
$$\frac{a + b * i}{c + d * i} = \frac{(a + b * i)(c - d * i)}{(c + d * i)(c - d * i)} = \frac{ac + bd}{c^2 + d^2} + \frac{bc - ad}{c^2 + d^2} * i$$

+ Các bài thực hành mức độ nâng cao.

Bài 1: Xây dựng interface Imanufacturer gồm có các phương thức:

```

interface IManufacturer {
    // phương thức trừu tượng để thêm thông tin liên hệ
    public void addContact(String detail);
    // phương thức trừu tượng để gọi nhà sản xuất
    public void callManufacturer(String phone);
    /**
     ** phương thức để tiến hành thanh toán
     */
    public void makePayment(float amount);
}

```

Bài 2: Xây dựng lớp **TwoWheeler1** được kế thừa từ hai giao diện **IVehicle**, **IManufacturer** và cài đặt các phương thức :

```
class TwoWheeler1 implements IVehicle, IManufacturer {  
    String ID;  
    // ID xe  
    String type; // kiểu xe  
    public TwoWheeler1(String ID, String type) {  
        this.ID = ID;  
        this.type = type;  
    }  
    @Override  
    public void start() {  
        System.out.println("Starting the " + type);  
    }  
    @Override  
    public void accelerate(int speed) {  
        System.out.println("Accelerating at speed:" + speed + " kmph");  
    }  
    @Override  
    public void brake() {  
        System.out.println("Applying brakes...");  
    }  
    @Override  
    public void stop() {  
        System.out.println("Stopping the " + type);  
    }  
    public void displayDetails() {  
        System.out.println("Vehicle No.: " + STATEID + " " + ID);  
        System.out.println("Vehicle Type.: " + type);  
    } // thực thi các phương thức của giao diện  
    @Override  
    public void addContact(String detail) {
```

```

        System.out.println("Manufacturer: " + detail);
    }
    @Override
    public void callManufacturer(String phone) {
        System.out.println("Calling Manufacturer @: " + phone);
    }
    @Override
    public void makePayment(float amount) {
        System.out.println("Payable Amount: $" + amount);
    }
}

```

Tạo lớp **TestVehicle1** và thể hiện của **TwoWheeler1**.

```

public class TestVehicle1 {
    public static void main(String[] args) {
        // thể hiện của lớp
        TwoWheeler1 objBike = new TwoWheeler1(args[0], args[1]);
        objBike.displayDetails();
        objBike.start();
        objBike.accelerate(Integer.parseInt(args[2]));
        objBike.brake();
        objBike.stop();
        objBike.addContact(args[3]);
        objBike.callManufacturer("0989555666");
        objBike.makePayment(50000);
    }
}

```